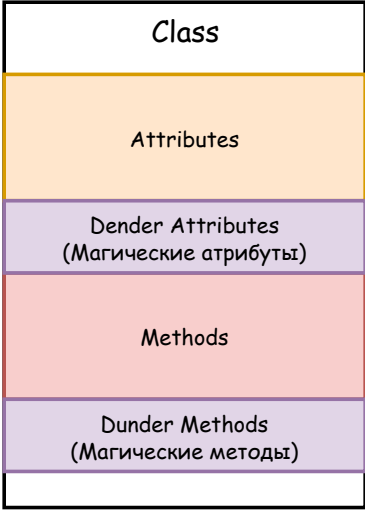




Dender Attributes
(Магические атрибуты)

- `__name__` : название функции, класса или модуля
- `__module__` : имя модуля для функции или класса
- `__doc__` строка документации для функции, класса или модуля.
- `__class__` : класс объекта
- `__dict__` : большинство объектов хранят свои атрибуты здесь.
- `__slots__` классы, использующие это, более эффективны с точки зрения использования памяти, чем классы, использующие `__dict__`
- `__match_args__` : классы могут определять кортеж, указывающий на значимость позиционных атрибутов, когда класс используется в структурном сопоставлении шаблонов (`match-case`)
- `__mro__` : порядок разрешения методов класса, используемый для поиска атрибутов и вызовов `super()`
- `__bases__` : прямые родительские классы класса
- `__file__` : файл, в котором определён объект модуля (хотя он не всегда присутствует!)
- `__wrapped__` : функции, отмеченные `functools.wraps` используют `this` для указания на исходную функцию
- `__version__` : обычно используется для обозначения версии пакета
- `__all__` : модули могут использовать это для настройки поведения `from my_module import *`
- `__debug__` : при запуске Python с `-O` устанавливается значение `False` и отключаются операторы `assert` в Python

- Функции имеют `__defaults__`, `__kwdefaults__`, `__code__`, `__globals__` и `__closure__`
- И функции, и классы имеют `__qualname__` и `__annotations__` `__type_params__`
- Классы имеют атрибуты `__static_attributes__` и `__firstlineno__` (Python 3.13+)
- Методы экземпляра имеют `__func__` и `__self__`
- Модули также могут иметь атрибуты `__loader__`, `__package__`, `__spec__`, и `__cached__`
- Пакеты имеют атрибут `__path__`
- Исключения составляют `__traceback__`, `__notes__`, `__context__`, `__cause__` и `__suppress_context__`
- Использование дескрипторов `__objclass__`
- Использование метаклассов `__classcell__`
- Модуль `weakref` в Python использует `__weakref__`
- Общие псевдонимы имеют `__origin__`, `__args__`, `__parameters__` и `__unpacked__`
- Модуль `sys` содержит `__stdout__` и `__stderr__`, которые указывают на исходные версии `stdout` и `stderr`

```
class Person:

    def __new__(cls, *args, **kwargs):
        print("new", args, kwargs)
        return super().__new__(cls)

    def __init__(self, name, age):
        self.name = name
        self.user_id = str(uuid.uuid4())
        self.age = age

    def __del__(self):
        print(f"Удаление объекта {self.user_id}")

st1 = Student("Anna", 30, course="Python")
```

